Session 14 Homework — Email Identity, a Workload-Driven Index, and Security Reasoning

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

A new requirement has arrived: email identity must be case-insensitive. `Nawar@Example.com`, `nawar@example.com`, and `NAWAR@EXAMPLE.COM` must all represent the exact same account — whether a user is registering for the first time or logging back in later with different casing than they originally used.

Goal

Update the taught registration/login flow so email identity is resolved consistently, no matter which casing is submitted, at every real boundary that resolves it.

What to build

1. Normalize email (trim + lowercase) when a new account registers, before the uniqueness check and before the account is persisted.
2. Confirm duplicate registration is correctly rejected even when the second attempt uses different casing than the first.
3. Normalize the submitted email the same way on login, before it is used to look up the account.
4. Confirm login succeeds regardless of which casing is submitted, as long as the password is correct.
5. Confirm the persisted `User.email` follows one single, consistent canonical format.
6. Confirm password behavior (hashing, verification) is completely unchanged.
7. Confirm the safe-response shape (no `passwordHash` anywhere) is completely unchanged.
8. Prove every one of the above with real requests.

CORE (required)

Update both `RegisterDto` and `LoginDto` so that email identity is resolved consistently — normalization must happen at EVERY boundary that resolves that identity, not only the one where the account was first created. Do not introduce new libraries or any future Auth concept (no JWT verification, no guards, no roles).

STRETCH (required)

A new product requirement has also arrived, separately: `GET /issues?status=todo` (status alone, no `projectId`) is now becoming a frequently-used query by itself. This session's own catch-up closed out the compound `{ projectId, status }` index — decide, from real measurement, whether that index set is still the right one once this NEW query shape starts mattering:

1. Run the real status-only query and inspect `explain("executionStats")`.
2. Record the real evidence: which stage, how many documents examined, how many index keys examined.
3. Explain, in writing, exactly why the current `{ projectId, status }` compound index does or does not help this specific query shape.
4. Decide whether a dedicated `{ status: 1 }` index is justified for this new workload.
5. If you decide to add it: add it, then measure the IDENTICAL query again.
6. Compare the before/after evidence, honestly, and keep or remove the new index based on what you actually measured.
7. Verify `projectId`-only and `projectId` + `status` queries still behave exactly as before.

CHALLENGE (required)

Write short, direct answers as comments directly above `AuthService`:

1. What is the difference between hashing and encryption?
2. Why should passwords be hashed instead of encrypted for this use case?
3. Why must `passwordHash` never be returned by the API?
4. A JWT is signed. Does that make its payload secret? Explain.
5. What kinds of values should never be placed in a JWT payload?

Rules

Allowed: everything class already used — `@Transform()`, `Schema.index()`, `explain("executionStats")`, `@ApiResponse`. No new syntax beyond what today's class already built.
Not yet: no checking/verifying a token on an incoming request, no route protection based on identity, no reading the logged-in user inside a handler, no protected routes, no roles/authorization, no interceptors, no Mongoose `ObjectId` / `ref` / `populate()`, no aggregation pipelines, no pagination.

Test it

BASH

```
curl -s -i -X POST localhost:3000/auth/register -H 'Content-Type: application/json' -d
'{"name":"...","email":"Mixed@Case.com","password":"..."}'
curl -s -i -X POST localhost:3000/auth/login -H 'Content-Type: application/json' -d
'{"email":"MIXED@case.COM","password":"..."}'
curl -s 'localhost:3000/issues?status=todo'
```

For STRETCH, in `mongosh`, run `explain("executionStats")` on the status-only query before you change anything, record the real numbers, make your decision, then (if you add the index) run the identical query again and compare.

Done when

- A mixed-case registration, a differently-cased duplicate rejection, the real canonical persisted email, and a differently-cased login success are all proven with real requests.
- Password behavior and the safe-response shape are both confirmed unchanged.
- Your STRETCH conclusion names the real `explain()` evidence you actually measured, and your decision is justified by that evidence, not assumed.
- Your CHALLENGE answers correctly distinguish hashing from encryption and explain what belongs (and does not belong) in a JWT payload.

Working time

About 60-90 minutes — the CORE code change is small (two `@Transform()` edits); most of the time is proving all four CORE cases, the STRETCH measurement, and writing precise answers.